Here we create an embedding for the user's query using the same model used for the stored questions. We will use the variable `queryEmbedding` next as an input to perform the most crucial part of the whole process, which is similarity searching.

SIMILARITY SEARCH

We compare the user's query embedding with the stored embeddings to find the most similar question. We use a simple `cosineSimilarity` function here to find out the most relevant question from the corpus.

Listing 5.11 Performing similarity search on the embeddings

```
let maxSimilarity = -1;
let mostRelevantQuestion = "";

for (const [question, storedEmbedding] of Object.entries(embeddingDB)) {
  const similarity =
cosineSimilarity(queryEmbedding, storedEmbedding);
  if (similarity > maxSimilarity) {
    maxSimilarity = similarity;
    mostRelevantQuestion = question;
  }
}
```

Sets up a variable to keep track of the highest cosine similarity found during the loop

Sets up a variable to store the question corresponding to the highest similarity found

Calculates the cosine similarity between the current stored embedding and the query embedding

If the current similarity is the highest seen so far, this block updates maxSimilarity and records the corresponding question as the most relevant one.

This `cosineSimilarity` function and the loop will allow you to identify the question in the knowledge base that is most similar to the user's query based on their embeddings. Here we only store the most similar question, but we can also tweak the algorithm to store the top 5 or 10 questions that have close similarity with the original query.

RESPONSE

The last part is simply returning the most relevant question matched to the user. This could also be part of the response as a list of relevant questions, or we can ask the AI model to write a nicely formatted output.

Listing 5.12 Returning the most relevant question

```
const relevantAnswer = answers[questions.indexOf(mostRelevantQuestion)];
console.log(`Most relevant question: ${mostRelevantQuestion}`);
console.log(`Relevant answer: ${relevantAnswer}`);
```

Of course, this process, while effective, can be enhanced in several ways. For instance, implementing the system could benefit from incremental learning, allowing it to update and refine its knowledge base continuously as new questions and answers emerge. Incorporating a feedback loop where users can rate the relevance of the